

1. A genealogia das linguagens não é uma escada de progresso. Explique essa afirmação e apresente dois fatores históricos que fazem uma linguagem influenciar outra sem necessariamente substituí-la.

R: A afirmação significa que a evolução das linguagens de programação não ocorre de forma linear, como se uma linguagem nova fosse sempre melhor e substituísse completamente a anterior. Diferentes linguagens continuam sendo usadas porque atendem a objetivos, sistemas e comunidades distintos, além de incorporarem características umas das outras.

2. Plankalkül não foi implementada em sua época. Ainda assim, por que ela é relevante para a história das linguagens? Cite três recursos antecipados por seu projeto e explique o valor de um deles.

R: Pois Plankalkül já possuía conceitos e ideias que só foram ser implementadas por linguagens muitos anos à frente de sua criação, como o uso de operações condicionais e controle de fluxo que incluía ideias semelhantes às estruturas de decisão usadas em linguagens modernas; Estruturas de dados como vetores e registros, permitindo organizar informações de maneira estruturada; Tipos de dados que previa diferentes formas de representar e manipular dados. Valor de um desses recursos: as estruturas de dados foram importantes porque permitem representar informações complexas de forma organizada, facilitando a criação de programas mais elaborados. Isso mostra que a Plankalkül já apresentava conceitos que seriam fundamentais para o desenvolvimento posterior das linguagens de programação.

4 - Explique por que o projeto Fortran precisou convencer programadores de que código traduzido podia competir com código de máquina escrito à mão. Relacione desempenho, custo de programação e adoção.

R: Na época, muitos programadores acreditavam que apenas o código produzido diretamente por especialistas conseguiria aproveitar ao máximo o computador. O principal desafio era o desempenho. Os computadores eram caros e tinham recursos limitados, então qualquer perda de velocidade causada pelo compilador poderia ser considerada inaceitável. O Fortran buscou demonstrar que seu compilador era capaz de gerar código eficiente, reduzindo a diferença de desempenho em relação ao código escrito à mão.

Ao mesmo tempo, havia uma grande vantagem no custo de programação. Escrever diretamente em linguagem de máquina ou assembly exigia muito conhecimento, tempo e trabalho, além de tornar os programas mais difíceis de manter. Com Fortran, os programadores podiam escrever instruções de maneira mais próxima da linguagem humana e deixar o compilador responsável pela tradução, aumentando significativamente a produtividade.

Isso estava diretamente relacionado à adoção da linguagem. Se o Fortran produzisse programas muito mais lentos, os programadores não teriam motivos para abandonar as técnicas tradicionais. Porém, ao demonstrar que era possível obter bom desempenho com muito menos esforço de programação, o projeto mostrou que o ganho de produtividade compensava o pequeno custo da tradução automática. Assim, o Fortran ajudou a

estabelecer a ideia de que linguagens de alto nível poderiam ser práticas e eficientes, contribuindo para sua ampla adoção e influenciando o desenvolvimento de outras linguagens de programação.

5. Lisp surgiu em um contexto diferente de Fortran. Compare os domínios, a representação de dados e o estilo de computação favorecido pelas duas linguagens.

R: Lisp surgiu em um contexto voltado principalmente para Inteligência Artificial e processamento simbólico, enquanto Fortran foi criada para aplicações científicas, especialmente cálculos numéricos. Fortran trabalhava principalmente com números, vetores e matrizes, priorizando a eficiência dos cálculos e do código gerado.

Na representação de dados, Fortran utilizava estruturas adequadas aos cálculos científicos, como vetores e matrizes. Já Lisp foi desenvolvida para manipular símbolos e listas, sendo sua estrutura básica formada por átomos e listas encadeadas. Essa representação facilitava o processamento de informações simbólicas, comum em aplicações de IA.

Quanto ao estilo de computação, Fortran seguia principalmente o paradigma imperativo, utilizando variáveis, atribuições, laços e operações numéricas. Lisp, por outro lado, foi projetada como uma linguagem funcional, na qual as computações são realizadas principalmente pela aplicação de funções aos argumentos, podendo utilizar recursão em vez de estruturas de repetição tradicionais.

Assim, enquanto Fortran favorecia cálculos numéricos eficientes para a computação científica, Lisp favorecia a manipulação de símbolos, listas e funções, sendo especialmente adequada ao processamento simbólico e à Inteligência Artificial.

7 - COBOL foi desenhada para processamento comercial. Mostre como domínio e público influenciaram sua legibilidade, seus registros e sua relação com FLOW-MATIC.

R: COBOL foi desenvolvida principalmente para o processamento comercial, especialmente para atividades relacionadas ao processamento de dados e à contabilidade. Por isso, seu projeto foi influenciado pela necessidade de ser uma linguagem fácil de utilizar e compreensível para um público mais amplo, incluindo pessoas da área de negócios e gerentes.

A legibilidade foi uma característica importante. O projeto de COBOL procurou utilizar o inglês o máximo possível, pois acreditava-se que isso facilitaria a programação e permitiria que os próprios gerentes conseguissem ler e compreender os programas. A linguagem também permitia nomes longos e significativos, utilizando hífen para separar palavras, tornando os comandos e identificadores mais próximos da linguagem natural.

Os registros também foram fundamentais para o domínio comercial. COBOL introduziu estruturas de dados hierárquicas, chamadas registros, que permitiam representar informações de negócios de forma organizada, como dados de funcionários, clientes e arquivos. A divisão de dados descrevia detalhadamente cada variável e os registros dos arquivos, incluindo informações como quantidade de dígitos e posição do ponto decimal. Essa característica tornou COBOL adequada para aplicações como relatórios contábeis.

A relação com FLOW-MATIC foi direta, pois ela foi a principal precursora de COBOL. Desenvolvida na UNIVAC, FLOW-MATIC já utilizava a ideia defendida por Grace Hopper de que programas de processamento de dados deveriam ser escritos usando sentenças em inglês. COBOL aproveitou e ampliou essa proposta, buscando criar uma linguagem comum para aplicações comerciais, independente de um fabricante específico.

Assim, o domínio comercial determinou a preocupação de COBOL com dados estruturados e relatórios, enquanto seu público influenciou a busca por uma linguagem mais legível e próxima do inglês. FLOW-MATIC serviu como uma das principais bases para essas ideias no desenvolvimento de COBOL.

1- Ada resultou de requisitos e projeto em grande escala. Analise como confiabilidade, tipos, pacotes e concorrência se relacionam ao domínio de sistemas críticos.

Confiabilidade

Em sistemas críticos, erros de software podem ter consequências graves. Por isso, Ada possui recursos que favorecem a detecção de erros ainda durante a compilação, reduzindo a possibilidade de falhas durante a execução. O tratamento de exceções também contribui para que situações anormais sejam tratadas de forma controlada.

Sistema de tipos

Ada possui um sistema de tipos forte, permitindo detectar incompatibilidades entre diferentes tipos de dados. Além disso, possibilita a criação de tipos específicos para representar conceitos diferentes.

Isso é importante em sistemas críticos porque ajuda a evitar operações inválidas e torna o programa mais previsível. Muitos erros que poderiam passar despercebidos em linguagens menos rigorosas podem ser identificados pelo compilador.

Pacotes

Os pacotes (packages) permitem dividir um sistema grande em módulos, separando sua interface de sua implementação.

Essa característica favorece o encapsulamento, a abstração e a manutenção do software. Em sistemas complexos, dividir o programa em componentes bem definidos facilita testes, manutenção e compreensão do código.

Concorrência

Ada possui mecanismos próprios para programação concorrente, permitindo que diferentes tarefas sejam executadas de maneira coordenada.

Isso é especialmente relevante em sistemas críticos e de tempo real, nos quais diversas atividades podem precisar acontecer simultaneamente, como monitoramento de sensores, controle de equipamentos e comunicação entre componentes.

2- Compare o papel dos objetos em Smalltalk, C++ e Java. Inclua na resposta o compromisso de C++ com C e a estratégia de portabilidade de Java.

1. Smalltalk

Em Smalltalk, o conceito de objeto é central para praticamente toda a linguagem. A linguagem foi projetada seguindo uma visão puramente orientada a objetos.

Tudo é tratado como objeto, e a comunicação entre objetos ocorre por meio do envio de mensagens. Dessa forma, não há uma separação tão forte entre elementos considerados "objetos" e elementos considerados apenas estruturas primitivas.

Isso torna o modelo conceitualmente uniforme e coloca a orientação a objetos no centro da linguagem.

2. C++

O C++ surgiu a partir da linguagem C, inicialmente como uma extensão de C com recursos de orientação a objetos.

Por isso, C++ precisou assumir um compromisso importante: adicionar orientação a objetos sem abandonar a compatibilidade e as características fundamentais de C.

Diferentemente de Smalltalk, C++ não exige que tudo seja objeto. A linguagem mantém recursos tradicionais de C, como:

- tipos primitivos;
- funções;
- ponteiros;
- estruturas;
- manipulação de memória;
- programação procedural.

Ao mesmo tempo, adiciona recursos como:

- classes;
- herança;
- polimorfismo;
- encapsulamento.

Assim, C++ pode ser utilizada tanto de maneira procedural quanto orientada a objetos.

3. Java

Java também utiliza fortemente o paradigma orientado a objetos, mas adotou uma estratégia diferente da de C++.

Java foi projetada para oferecer portabilidade entre diferentes plataformas. Seu princípio ficou associado à ideia de que o mesmo programa poderia ser executado em diferentes sistemas por meio da Java Virtual Machine (JVM).

O código Java é compilado para bytecode, que não é diretamente específico de um determinado processador. A JVM de cada plataforma interpreta ou compila esse bytecode para o ambiente correspondente.

3- A primeira aplicação de Java não foi a Web, mas a Web impulsionou sua adoção. Explique como mudanças de contexto podem reposicionar uma linguagem.

O Java foi desenvolvido inicialmente com a ideia de ser utilizado em dispositivos eletrônicos e sistemas embarcados, dentro do projeto Green, da Sun Microsystems. Entretanto, com a expansão da World Wide Web, percebeu-se que suas características, especialmente a portabilidade proporcionada pela máquina virtual Java (JVM), poderiam ser muito úteis para aplicações distribuídas na Web.

A possibilidade de compilar o programa para bytecode e executá-lo em diferentes plataformas por meio da JVM tornou o Java atraente para a Web, pois reduzia a dependência do sistema operacional e do hardware específico. Os applets Java foram uma das primeiras formas de exploração dessa ideia nos navegadores.

Assim, o sucesso do Java não ocorreu simplesmente porque sua finalidade original era a Web. Uma mudança no contexto tecnológico criou uma nova oportunidade de aplicação para características que a linguagem já possuía. Posteriormente, o Java deixou de estar associado apenas aos navegadores e passou a ser amplamente utilizado em servidores, sistemas corporativos, aplicações móveis e outros ambientes.

Isso mostra uma ideia importante apresentada por Sebesta: a evolução e a adoção de uma linguagem dependem não apenas de suas características técnicas, mas também das necessidades e tendências do ambiente tecnológico em que ela é utilizada.

4- Compare Perl, JavaScript, PHP, Python, Ruby e Lua usando três eixos: domínio inicial, estruturas de dados e estratégia de implementação. Evite concluir que todas são iguais por serem chamadas de scripting.

1. Domínio inicial

- **Perl** surgiu fortemente associado ao processamento de texto e administração de sistemas.
- **JavaScript** foi criado inicialmente para fornecer scripts em páginas Web, permitindo interação dinâmica no navegador.
- **PHP** surgiu direcionado à criação de páginas Web dinâmicas, sendo executado principalmente no servidor.

- **Python** foi projetado como uma linguagem de uso geral, com forte preocupação com legibilidade e produtividade, sendo utilizada posteriormente em diversos domínios.
- **Ruby** foi criada como uma linguagem de uso geral, com foco em simplicidade, produtividade e programação orientada a objetos, ganhando grande popularidade no desenvolvimento Web.
- **Lua** teve como característica marcante ser uma linguagem embutível/extensível, destinada a ser integrada a outras aplicações.

2. Estruturas de dados

Perl possui escalares, arrays e hashes como estruturas fundamentais. Os hashes são particularmente importantes para representar associações entre chaves e valores.

JavaScript utiliza principalmente objetos e arrays. Os objetos permitem representar estruturas associativas e complexas, enquanto arrays são utilizados para coleções ordenadas.

PHP possui como uma de suas estruturas mais características o array, que funciona de maneira bastante flexível, podendo assumir comportamento semelhante tanto ao de uma lista quanto ao de um mapa associativo.

Python apresenta estruturas de alto nível bastante distintas:

- **list** → coleção ordenada e mutável;
- **tuple** → coleção ordenada e imutável;
- **dict** → associação chave/valor;
- **set** → conjunto de elementos únicos.

Ruby trabalha fortemente com objetos e possui estruturas como **Array** e **Hash**.

Lua possui uma característica particularmente interessante: sua estrutura fundamental é a tabela (*table*). Uma tabela pode funcionar como array, estrutura associativa ou até servir como base para representar outras estruturas mais complexas.

3. Estratégia de implementação

Perl: o código-fonte passa por um processo de análise/compilação interna e depois é executado pelo interpretador.

JavaScript: historicamente foi associado à interpretação pelo navegador, mas os motores modernos utilizam técnicas de compilação JIT (Just-in-Time). Portanto, classificá-lo simplesmente como "interpretado" é uma simplificação.

PHP: o código é transformado em uma representação intermediária (opcode) executada pelo motor da linguagem. Implementações modernas também podem utilizar JIT.

Python: na implementação mais conhecida, o código-fonte é convertido para bytecode, que posteriormente é executado pela máquina virtual do Python.

Ruby: implementações modernas, como o MRI/CRuby, também utilizam uma representação intermediária e uma máquina virtual, podendo empregar JIT.

Lua: normalmente transforma o código em bytecode, posteriormente executado pela máquina virtual Lua

5. Diferencie XSLT e JSP quanto a entrada, processamento e saída. Por que ambas podem ser chamadas de linguagens híbridas de marcação e programação?

1. XSLT

- Entrada: recebe principalmente um documento XML.
- Processamento: utiliza regras de transformação para analisar a estrutura do XML e determinar como seus elementos devem ser transformados.
- Saída: normalmente produz outro documento, como HTML, XML ou texto.

O XSLT é, portanto, voltado principalmente para a transformação de documentos estruturados.

2. JSP

- Entrada: recebe uma página JSP contendo HTML e elementos de programação, além de dados fornecidos pela aplicação ou pelo usuário.
- Processamento: o servidor processa a página, executando as partes dinâmicas e combinando seus resultados com a marcação HTML.
- Saída: normalmente produz HTML, que é enviado ao navegador do usuário.

As duas podem ser chamadas de linguagens híbridas de marcação e programação porque combinam dois aspectos:

- Marcação: utilizam elementos que estruturam e descrevem o conteúdo do documento.
- Programação/processamento: possuem mecanismos para realizar operações, tomar decisões, selecionar dados e produzir uma saída dinamicamente.

A diferença principal é que XSLT é focada na transformação de documentos, enquanto JSP é focada na geração dinâmica de páginas Web. Portanto, embora ambas misturem marcação e processamento, seus domínios de aplicação e modelos de execução são diferentes.